

API REFERENCE — Modulo Booking

Backend Gestionale B&B Cosenza Versione API v1 · Documento aggiornato al 20 settembre 2026 · Copertura: Step A → F

Come usare questo documento

Serve a tre scopi, e ogni sezione è pensata per uno di essi.

Per il collaudo manuale: la sezione *Endpoint* descrive ogni rotta con i suoi codici di risposta e un esempio `curl` pronto da incollare. La *Checklist di verifica* in fondo è la sequenza da seguire a sviluppo concluso.

Per lo sviluppo frontend: la sezione *Schemi* elenca ogni campo con tipo, vincoli e obbligatorietà. La sezione *Meccanismi trasversali* spiega cosa il client deve inviare oltre ai dati veri e propri (token del preventivo, captcha, honeypot) e come interpretare gli errori.

Per chi riprende lo sviluppo: la *Logica* di ciascun endpoint dice cosa accade dietro la chiamata e quali test la coprono.

Questo documento viene aggiornato a ogni modifica del modulo. Il *Changelog* in fondo traccia cosa è cambiato e quando.

1. Convenzioni generali

1.1 Base URL e versioning

```
http://localhost:8000/api/v1
```

Tutte le rotte del modulo sono sotto `/api/v1/bookings`.

1.2 Formato delle date

Campo	Formato	Esempio
Date di soggiorno (<code>check_in</code> , <code>check_out</code>)	<code>YYYY-MM-DD</code> (solo data)	<code>2026-12-01</code>
Istanti (<code>hold_expires_at</code> , <code>confirmed_at</code> , ...)	ISO 8601 con fuso, in UTC	<code>2026-12-01T14:30:00Z</code>

⚠ Nota per il frontend. Le date di soggiorno **non** sono timestamp. `check_out` è il giorno di partenza ed è **escluso** dal soggiorno: `2026-12-01 → 2026-12-03` sono **due notti**. Prima dello Step A questi campi erano timestamp completi: se esiste codice Angular che li tratta come tali, va adeguato.

1.3 Importi

Tutti gli importi sono **stringhe decimali con due cifre** (`"200.00"`), non numeri in virgola mobile. Serializzarli come `float` in JavaScript introduce errori di arrotondamento sui totali.

La valuta è in `currency` (attualmente sempre `"EUR"`).

1.4 Formato degli errori

Ogni errore ha la stessa forma:

```
{
  "message": "Testo leggibile, già in italiano, mostrabile all'utente"
}
```

Gli errori di validazione hanno un campo aggiuntivo:

```
{
  "message": "La password deve contenere almeno una maiuscola",
  "details": [
    { "type": "value_error", "loc": ["body", "password"], "msg": "..." }
  ]
}
```

`details` serve al frontend per evidenziare il campo in errore (`loc` contiene il percorso). **Il valore rifiutato non viene mai restituito**: `details` non contiene la chiave `input`, per evitare che una password o un dato personale torni indietro nella risposta.

1.5 Codici di stato e loro significato

Codice	Quando	Cosa deve fare il client
200	Operazione riuscita	—
201	Prenotazione creata	—
400	Token di conferma o cancellazione non valido	Invitare a richiedere un nuovo link
401	Sessione assente o scaduta	Reindirizzare al login
403	Ruolo insufficiente	Nascondere la funzione
404	Risorsa inesistente, o non di proprietà dell'utente	Messaggio generico
409	Conflitto: slot occupato, transizione non ammessa, già confermata	Mostrare il messaggio e proporre una nuova ricerca
410	Blocco temporaneo scaduto	Invitare a ripetere la prenotazione
422	Dati non validi, preventivo scaduto, captcha fallito	Evidenziare il campo tramite <code>details</code>
429	Troppe richieste	Attendere i secondi indicati in <code>Retry-After</code>
402	Pagamento richiesto o fallito (<i>dallo Step G</i>)	—

1.6 Mappa eccezione → codice

Il backend non espone mai stack trace. Ogni eccezione di dominio deriva da `AppException` e porta con sé il proprio codice HTTP; un **unico handler** globale le traduce in risposta.

Eccezione	Codice	Messaggio predefinito
<code>EntityNotFound</code>	404	Entity not found
<code>EntityAlreadyExists</code>	409	Entity already exists
<code>EntityInUse</code>	409	L'elemento è utilizzato da altri dati e non può essere eliminato
<code>RoomNotAvailable</code>	409	Una o più camere non sono disponibili per le date richieste
<code>InvalidBookingStatusTransition</code>	409	L'operazione non è consentita nello stato attuale
<code>BookingNotCancellable</code>	409	La prenotazione non può più essere cancellata
<code>BookingHoldExpired</code>	410	Il tempo per completare la prenotazione è scaduto
<code>InvalidBookingToken</code>	400	Il link utilizzato non è valido o è già stato usato
<code>InvalidDateRange</code>	422	L'intervallo di date indicato non è valido
<code>InvalidGuestCount</code>	422	Il numero di ospiti non è compatibile con le camere
<code>InvalidQuoteToken</code>	422	Il preventivo non è più valido
<code>CaptchaValidationFailed</code>	422	Verifica anti-bot non superata
<code>ConcurrentModification</code>	409	La prenotazione è stata modificata da un altro operatore
<code>RateLimitExceeded</code>	429	Troppe richieste: riprova più tardi
<code>InvalidFileType</code> / <code>InvalidFileSize</code>	422	—
<code>StaleDataError</code> (SQLAlchemy)	409	Il dato è stato modificato da un'altra operazione

2. Autenticazione e autorizzazione

2.1 Come funziona

Il progetto usa **JWT veicolati da cookie** `HttpOnly`, non header `Authorization`. Il token non è leggibile da JavaScript, il che elimina il furto per Cross-Site Scripting.

Login su `POST /api/v1/auth/token` con form data (`username` = email, `password`). La risposta imposta due cookie:

Cookie	Durata	Attributi
<code>access_token</code>	breve (<code>access_token_expire_minutes</code>)	<code>HttpOnly</code> , <code>SameSite=lax</code> , <code>Path=</code>
<code>refresh_token</code>	lunga (<code>refresh_token_expire_minutes</code>)	<code>HttpOnly</code> , <code>SameSite=lax</code> , <code>Path=</code>

⚠ **Frontend:** tutte le chiamate autenticate devono usare `withCredentials: true` (Angular `HttpClient`) o `credentials: 'include'` (fetch). Senza, il cookie non viene inviato e la risposta è `401`.

⚠ **Produzione:** i cookie hanno attualmente `secure=False`. Va portato a `True` con HTTPS attivo. (Voce di backlog, non ancora risolta.)

2.2 Livelli di accesso

Livello	Dipendenza	Comportamento
Pubblico	nessuna	Chiunque, anche non registrato
Autenticato	<code>Depends(get_current_user)</code>	Cookie <code>access_token</code> valido; altrimenti <code>401</code>
Amministrativo	<code>Depends(is_admin_user)</code>	Utente con <code>role = admin</code> ; altrimenti <code>403</code>

`is_admin_user` è un `RoleChecker(allowed_roles=[UserRole.admin])` .

2.3 Cosa ha aggiunto il modulo Booking

Nessuna modifica al meccanismo di autenticazione esistente. Il modulo introduce però un secondo sistema di credenziali, **indipendente e non sostitutivo**:

- **Token monouso di prenotazione** — consentono a un ospite *non registrato* di confermare o cancellare la propria prenotazione senza avere un account. Non sono sessioni: valgono per una sola operazione su una sola prenotazione.

I due sistemi non si incrociano: un token di prenotazione non autentica l'utente, e una sessione valida non sostituisce il token nel link ricevuto via email.

3. Meccanismi trasversali introdotti

Questa sezione descrive tutto ciò che è stato aggiunto al progetto a supporto del modulo Booking.

3.1 Rate limiting

File: `src/security/rate_limiter.py`

Finestra scorrevole: per ogni chiave si conservano gli istanti delle richieste recenti e si scartano quelle uscite dalla finestra. A differenza del conteggio a finestra fissa, impedisce il raddoppio del limite a cavallo di due intervalli.

Limiti attivi (configurabili da `.env`):

Ambito	Limite	Finestra	Chiave
<code>GET /availability</code>	20	1 minuto	IP
<code>POST /quote</code>	20	1 minuto	IP
<code>POST /</code> (creazione)	5	1 ora	IP
<code>POST /</code> (creazione)	3	24 ore	email
<code>POST /confirm</code> , <code>/cancel</code>	10	1 ora	IP
<code>POST /lookup</code>	10	1 ora	IP

I limiti per IP ed email sono complementari: un attaccante con molti indirizzi IP resta vincolato sul numero di prenotazioni riconducibili alla stessa email, e viceversa.

Risposta al superamento: `429` con header `Retry-After` in secondi.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 42
```

```
{"message": "Troppe richieste: riprova più tardi"}
```

Due limiti noti, da tenere presenti:

1. Il **contatore è per processo**. Con `uvicorn --workers 4` ogni worker ha il proprio, quindi il limite effettivo è il quadruplo. Su un singolo worker il conteggio è esatto. L'interfaccia `RateLimitBackend` consente di sostituire lo storage con Redis senza toccare i router.
2. **X-Forwarded-For è ignorato per impostazione predefinita** (`trusted_proxy_count = 0`). Quell'header lo scrive il client: fidarsene senza un proxy che lo riscriva permetterebbe a chiunque di cambiare IP a ogni richiesta e azzerare il rate limiting. Quando l'app andrà dietro Nginx o Cloudflare, impostare `TRUSTED_PROXY_COUNT=1`.

3.2 Captcha — Cloudflare Turnstile

File: `src/security/captcha.py`

Verificato solo su `POST /api/v1/bookings/` (creazione pubblica). Scelto al posto di reCAPTCHA perché non profila gli utenti e non richiede un consenso ai cookie aggiuntivo.

- Disattivato per impostazione predefinita (`CAPTCHA_ENABLED=false`): in sviluppo non serve configurare nulla.
- Il frontend inserisce il widget Turnstile e invia il token risultante nel campo `captcha_token`.
- Token assente o rifiutato → `422`.

Comportamento in caso di disservizio: se Cloudflare non risponde o va in timeout (5 secondi), la richiesta **viene lasciata passare** e l'evento registrato come warning. Bloccare tutte le prenotazioni durante un disservizio di terze parti costerebbe incassi reali; le poche richieste automatiche che passassero creerebbero prenotazioni non confermate, che scadono da sole in quindici minuti.

3.3 Honeypot

Il campo `website` di `GuestBookingCreateSchema` è una trappola: va reso **invisibile via CSS** nel form e mai compilato da un utente reale. Se arriva valorizzato, la richiesta è automatizzata e viene respinta con `422`.

Costa nulla e intercetta i bot che compilano ogni campo del form.

3.4 Preventivo firmato (`quote_token`)

File: `src/security/quote_token.py`

Il client non invia mai un prezzo. Il flusso pubblico è a due passi per costruzione:

1. `POST /quote` calcola il totale e lo restituisce racchiuso in un JWT firmato, valido 15 minuti;
2. `POST /` accetta esclusivamente quel token.

Il token contiene camere, date, numero di ospiti, opzione di pagamento e totale. Protegge da due cose: la **manomissione del prezzo**, perché la firma lo rende immodificabile, e lo **spostamento della selezione**, perché non si possono sostituire le camere dopo il calcolo.

È firmato con lo stesso segreto dell'autenticazione ma con un claim `type` diverso: un access token non può essere speso come preventivo, né viceversa.

Difesa in profondità: anche con un token valido, il server **ricalcola** il totale alla creazione e rifiuta con `422` se diverge. La firma dice "questo prezzo l'ho emesso io", non "questo prezzo è ancora corretto".

3.5 Token monouso di prenotazione

File: `src/security/booking_tokens.py`

Generati con `secrets.token_urlsafe(32)` — 256 bit di entropia. Nel database finisce **solo l'hash SHA-256**, mai il valore in chiaro: una compromissione del database non consente di confermare o cancellare prenotazioni altrui.

Scopo	Uso
<code>CONFIRM_EMAIL</code>	Conferma della prenotazione (solo <code>PAY_ON_ARRIVAL</code>)
<code>MANAGE</code>	Gestione della prenotazione
<code>CANCEL</code>	Cancellazione

Sono **monouso**: al primo utilizzo `used_at` viene valorizzato e i tentativi successivi respinti.

⚠ Il token viaggia nel **body**, mai in **query string**. Un token nell'URL finisce negli access log, nell'header `Referer` e nella cronologia del browser. Il link nell'email punta al **frontend** (`{frontend_base_url}/booking/confirm?token=...`), e la SPA lo inoltra al backend con una `POST`.

3.6 Esposizione temporanea del token in sviluppo

Finché l'invio email non esiste (Step F), `POST /` restituisce il token di conferma nel campo `confirmation_token`, **ma solo quando** `EMAIL_ENABLED=false`. Attivando l'invio email il campo torna `null` automaticamente.

C'è un test che lo verifica, così l'affordance si spegne da sé senza che nessuno debba ricordarsene.

3.7 Gestione centralizzata degli errori

File: `src/exception/exception_handler.py`

Un **unico handler** registrato su `AppException` copre tutte le eccezioni di dominio, presenti e future: Starlette risolve percorrendo l'MRO dell'eccezione. Aggiungere un nuovo errore richiede una sola classe, non anche un handler.

Correzione importante introdotta allo Step B: gli errori di validazione producevano un `500` invece di un `422`, perché Pydantic v2 inserisce l'**oggetto eccezione vivo** nella chiave `ctx` e `json.dumps` non sa serializzarlo. Ora il contenuto di `ctx` viene convertito in stringa e le chiavi `input` e `url` rimosse.

3.8 Email transazionali

Quattro messaggi, ciascuno in versione HTML e testo semplice.

Template	Quando parte	Cosa contiene
<code>booking_pending</code>	Creazione con conferma richiesta	Link di conferma, scadenza del blocco
<code>booking_confirmed</code>	Conferma, o creazione già confermata	Riepilogo, link di gestione , termine gratuito
<code>booking_cancelled</code>	Annullamento, dall'ospite o dal back-office	Riepilogo, eventuale penale
<code>booking_expired</code>	Sweeper	Avviso, invito a riprenotare

I link puntano alla SPA, non al backend: `{FRONTEND_BASE_URL}/prenotazione/conferma?token=...` e `/prenotazione/gestisci?token=...`. Il frontend legge il token dalla query string e lo inoltra nel **body di una POST** al

backend. Il token non deve mai comparire in un URL del backend, dove finirebbe negli access log e nell'header `Referer`.

L'invio avviene dopo il commit, tramite `BackgroundTasks`. Non è un dettaglio implementativo: spedire dentro la transazione significherebbe, in caso di rollback, annunciare all'ospite una prenotazione che non esiste — e quella email non si richiama indietro.

Un invio fallito non fa fallire la richiesta. Se il server SMTP è irraggiungibile la prenotazione resta valida e l'evento finisce nei log. Stessa logica del captcha: rifiutare prenotazioni durante un disservizio di terze parti costerebbe incassi veri.

Nei log finiscono codice prenotazione, template, esito e destinatario mascherato (`m***@example.com`). **Non** il token, **non** il corpo del messaggio, **non** l'indirizzo completo.

Escaping. I template HTML hanno l'autoescape attivo. Il nome dell'ospite arriva da un form pubblico e finisce in un messaggio spedito con il mittente della struttura: senza escaping, chiunque potrebbe iniettare markup e link in una email che *sembra* del B&B.

Con `EMAIL_ENABLED=false` i messaggi vengono scritti nei log invece che spediti, link compreso. Perché siano visibili serve che i log applicativi abbiano una destinazione: `configure_logging()` in `main.py` attacca un handler al sottoalbero `src`. Senza, uvicorn configura solo i propri logger e ogni riga del progetto viene scartata in silenzio — il canale console sembrerebbe non funzionare. È l'unico punto del sistema in cui un token in chiaro finisce in un log, ed è attivo esattamente quando le email sono spente, cioè in sviluppo.

Nessuna email su `PENDING_PAYMENT`. Una prenotazione `PAY_NOW` nasce in attesa di incasso: l'ospite è ancora sulla pagina di pagamento e non c'è nulla da comunicargli. La notifica parte quando il webhook Stripe conferma (Step G).

3.9 Sweeper delle scadenze

Porta a `EXPIRED` le prenotazioni temporanee il cui blocco è scaduto. Parte con l'applicazione (`lifespan`) e gira ogni `SWEeper_INTERVAL_SECONDS` su lotti di `SWEeper_BATCH_SIZE`.

È l'unico proprietario della transizione a `EXPIRED`. In nessun altro punto del codice una prenotazione diventa `EXPIRED`.

Il sistema resta corretto anche se non gira. Gli slot tornano prenotabili per altre due vie: le query di disponibilità scartano i pending con blocco scaduto, e la *just-in-time expiration* disattiva le righe camera durante la creazione successiva sulle stesse date. Senza sweeper mancano la pulizia degli stati e l'avviso all'ospite, non la correttezza.

Con più worker uvicorn ne parte uno per processo. Le passate si dividono il lavoro grazie a `FOR UPDATE ... SKIP LOCKED`: chi arriva secondo salta le righe già prese in carico invece di aspettarle.

Arretrati. Se lo sweeper resta fermo a lungo, al riavvio sistema tutti gli stati ma **avvisa solo** le scadenze più recenti di `SWEeper_NOTIFY_MAX_AGE_HOURS` (default 24). Spedire centinaia di email su richieste dimenticate da giorni è un modo efficace per farsi segnalare come spam.

Disattivabile con `SWEeper_ENABLED=false`, per pilotarlo da uno scheduler esterno tramite l'endpoint 17.

4. Endpoint

4.1 Prospetto

#	Metodo	Rotta	Accesso	Rate limit	Captcha
1	GET	/api/v1/bookings/availability	Pubblico	20/min IP	no
2	POST	/api/v1/bookings/quote	Pubblico	20/min IP	no
3	POST	/api/v1/bookings/	Pubblico	5/h IP + 3/g email	sì
4	POST	/api/v1/bookings/confirm	Pubblico	10/h IP	no
5	POST	/api/v1/bookings/cancel	Pubblico	10/h IP	no
6	POST	/api/v1/bookings/lookup	Pubblico	10/h IP	no
7	GET	/api/v1/bookings/me	Autenticato	—	no
8	POST	/api/v1/bookings/me	Autenticato	—	no
9	POST	/api/v1/bookings/me/{code}/cancel	Autenticato	—	no

Area amministrativa, sotto `/api/v1/admin/bookings` — tutti con accesso **admin**:

#	Metodo	Rotta	Scopo
10	GET	/	Elenco filtrato e paginato
11	GET	/ {booking_id}	Dettaglio completo con timeline
12	POST	/	Creazione per conto di terzi
13	PATCH	/ {booking_id}	Modifica date, camere, ospiti, note
14	POST	/ {booking_id}/status	Transizione di stato
15	POST	/ {booking_id}/payment	Registrazione incasso manuale
16	POST	/ {booking_id}/extend-hold	Proroga del blocco temporaneo
17	POST	/sweep-expired	Esecuzione immediata dello sweeper delle scadenze

1 • GET /api/v1/bookings/availability

Camere disponibili nell'intervallo richiesto.

Accesso	Pubblico
Rate limit	20 richieste/minuto per IP
Query	<code>check_in</code> (date, obbligatorio) · <code>check_out</code> (date, obbligatorio) · <code>guest_count</code> (int ≥ 1, default 1)
Risposta	<code>AvailabilityResponseSchema</code>

Codici: `200` · `422` date non valide · `429` limite superato

Logica. Recupera le camere abilitate e sottrae quelle occupate nell'intervallo. Una camera risulta occupata quando esiste una riga attiva con date sovrapposte e la prenotazione è in uno stato occupante — per gli stati temporanei, solo se il blocco non è ancora scaduto.

Non filtra per capienza, di proposito: nasconderebbe la possibilità di prenotare due camere doppie per quattro persone, che in un B&B è il caso normale delle famiglie. Restituisce invece tutte le camere libere con il flag `fits_all_guests`, più le combinazioni utili.

Le tre modalità di prenotazione sono servite da questa singola chiamata:

Modalità	Come ottenerla dal frontend
Camera singola sufficiente	Filtrare <code>rooms</code> su <code>fits_all_guests == true</code>
Scelta manuale	Presentare <code>rooms</code> per intero — la somma delle capienze è verificata al preventivo
Combinazioni suggerite	Usare <code>suggested_combinations</code>

Le combinazioni sono minimali: se togliendo una camera gli ospiti ci starebbero ancora, la proposta non viene restituita. Con una doppia, un'altra doppia e una quadrupla, per 4 persone si ottengono due proposte — la quadrupla da sola, oppure le due doppie — e non le varianti ridondanti. Ordinate per numero di camere, poi prezzo, poi capienza sprecata. Massimo 10 proposte.

Test: `test_flusso_completo_availability_quote_create_confirm`, `test_availability_marca_le_camere_che_bastano_da_sole`, `test_availability_propone_combinazioni`, `test_availability_date_incoerenti_rifiutate`, `test_rate_limit_con_retry_after`, più l'intero file `test_availability_combinations.py` (11 test sulla logica pura).

```
curl -i "http://localhost:8000/api/v1/bookings/availability?check_in=2026-12-01&check_out=2026-12-03&
```

2 • `POST /api/v1/bookings/quote`

Preventivo firmato per una selezione di camere.

Accesso	Pubblico
Rate limit	20 richieste/minuto per IP
Body	<code>BookingQuoteRequestSchema</code>
Risposta	<code>BookingQuoteResponseSchema</code>

Codici: `200` • `404` camera inesistente • `409` camera occupata o disabilitata • `422` date non valide, ospiti oltre la capienza • `429`

Logica. Carica le camere, verifica che esistano e siano abilitate, controlla che la somma delle capienze copra gli ospiti, verifica la disponibilità, calcola il prezzo ed emette il token firmato.

È di sola lettura: non blocca nulla e non crea righe. La disponibilità viene verificata per dare all'utente un errore immediato invece di farglielo scoprire dopo aver compilato i propri dati, ma **la garanzia resta al momento della prenotazione:** fra il preventivo e la conferma qualcun altro può sempre arrivare prima.

Lo sconto per pagamento online (10%) è applicato qui e incluso nel token.

Test: `test_flusso_completo_...`, `test_preventivo_su_camera_inesistente`, e l'intero `test_pricing_service.py` (18 test su calcoli, arrotondamento e token).

```
curl -i -X POST http://localhost:8000/api/v1/bookings/quote \
-H "Content-Type: application/json" \
-d '{"check_in":"2026-12-01","check_out":"2026-12-03","guest_count":2,
    "room_ids":["<UUID_CAMERA>"],"payment_option":"PAY_ON_ARRIVAL"}'
```

3 • `POST /api/v1/bookings/`

Crea una prenotazione come ospite non registrato.

Accesso	Pubblico
Rate limit	5/ora per IP e 3/giorno per email
Captcha	Sì (se <code>CAPTCHA_ENABLED=true</code>)
Body	<code>GuestBookingCreateSchema</code>
Risposta	<code>BookingCreatedSchema</code> — 201

Codici: 201 • 404 camera inesistente • 409 slot occupato • 410 — • 422 preventivo scaduto o manomesso, honeypot, condizioni non accettate, captcha fallito • 429

Logica — l'operazione più delicata del modulo. Tutto dentro una sola transazione:

1. verifica del `quote_token` (firma e scadenza);
2. caricamento camere, controllo esistenza e abilitazione;
3. controllo capienza totale $\geq$ ospiti;
4. **ricalcolo del prezzo** e confronto con il token → 422 se diverge;
5. controllo del numero di prenotazioni in attesa per quella email;
6. **lock pessimistico** sulle camere, ordinato per id (nessun deadlock possibile);
7. **just-in-time expiration**: libera gli slot il cui blocco è scaduto;
8. verifica disponibilità → 409 con l'elenco delle camere non più libere;
9. generazione del codice prenotazione univoco;
10. creazione della prenotazione e delle righe camera, con prezzo congelato;
11. `flush` → **qui scatta l'exclusion constraint del database**: se due richieste simultanee arrivano insieme, una sola passa;
12. riga di storico e token di conferma.

Stato iniziale:

Opzione di pagamento	Stato	Token di conferma
<code>PAY_ON_ARRIVAL</code>	<code>PENDING_CONFIRMATION</code>	sì
<code>PAY_NOW</code>	<code>PENDING_PAYMENT</code>	no — il pagamento è la verifica d'identità

Lo slot resta bloccato **15 minuti** (`hold_expires_at`). Scaduto quel termine senza conferma, torna prenotabile e lo sweeper porta la prenotazione a `EXPIRED`.

Email: `booking_pending` con `PAY_ON_ARRIVAL`. Con `PAY_NOW` **nessuna email**: l'ospite è ancora sulla pagina di pagamento e la notifica parte dal webhook Stripe (Step G).

Test: `test_flusso_completo_...`, `test_slot_occupato_risponde_409`, `test_la_creazione_manda_una_sola_email_di_conferma`, `test_il_pagamento_online_non_manda_ancora_nulla`, `test_un_canale_email_guasto_non_impedisce_la_prenotazione`, `test_honeypot_compilato_rifiutato`, `test_condizioni_non_accettate_rifiutate`, `test_token_di_conferma_non_esposto_con_email_attiva`, più i 14 test di `test_booking_service.py` fra cui `test_due_prenotazioni_concorrenti_una_sola_vince`, ripetuto 10 volte.

```
curl -i -X POST http://localhost:8000/api/v1/bookings/ \
-H "Content-Type: application/json" \
-d '{"quote_token": "<TOKEN_DAL_PREVENTIVO>",
  "guest": {"firstname": "Mario", "lastname": "Rossi",
    "email": "mario@example.com", "phone_number": "3331234567"},
  "accept_terms": true}'
```

4 • `POST /api/v1/bookings/confirm`

Conferma tramite il token ricevuto per email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	<code>BookingConfirmSchema</code>
Risposta	<code>BookingPublicSchema</code>

Codici: `200` · `400` token inesistente o già speso · `409` già confermata · `410` blocco scaduto · `429`

Logica. Cerca il token per hash, verifica che sia del tipo corretto e non usato, controlla che né il token né il blocco siano scaduti, esegue la transizione a `CONFIRMED`, azzerà `hold_expires_at`, calcola il termine di cancellazione gratuita, marca il token come speso e invalida gli altri della stessa prenotazione.

Doppio clic sul link: se il token è già speso e la prenotazione è `CONFIRMED`, la risposta è `409` con *"La prenotazione è già stata confermata"* — distinto dal generico "link non valido", per non far dubitare l'ospite che la conferma sia andata a buon fine.

Blocco scaduto: `410`. Lo stato resta `PENDING_CONFIRMATION` finché non passa lo sweeper; lo slot è comunque già tornato prenotabile.

Emette il token di gestione. Alla conferma viene creato un token `MANAGE`, valido fino alla data di partenza, che viaggia nel link dell'email di riepilogo. Per un ospite non registrato è **l'unico modo di annullare in autonomia**: non ha un account, e il `lookup` è in sola lettura.

Email: `booking_confirmed`, con il link di gestione.

Test: `test_flusso_completo_...`, `test_doppia_conferma_risponde_409`, `test_conferma_completa_il_ciclo`, `test_doppia_conferma segnalata`, `test_conferma_dopo_la_scadenza_rifiutata`, `test_la_conferma_manda_il_riepilogo_con_il_link_di_gestione`.

```
curl -i -X POST http://localhost:8000/api/v1/bookings/confirm \
-H "Content-Type: application/json" -d '{"token": "<TOKEN_DI_CONFERMA>"}'
```

5 • `POST /api/v1/bookings/cancel`

Cancellazione tramite link ricevuto per email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	<code>BookingCancelSchema</code>
Risposta	<code>BookingPublicSchema</code>

Codici: `200` • `400` token non valido • `409` non cancellabile • `429`

Logica. Accetta token di tipo `CANCEL` o `MANAGE`. Verifica che la cancellazione sia ammessa:

Situazione	Esito
Stato <code>PENDING_*</code>	Consentita
<code>CONFIRMED</code> + <code>PAY_ON_ARRIVAL</code> entro il termine	Consentita, gratuita
<code>CONFIRMED</code> + <code>PAY_ON_ARRIVAL</code> oltre il termine	<code>409</code> — invito a contattare la struttura
<code>CONFIRMED</code> + <code>PAY_NOW</code>	<code>409</code> — non rimborsabile, invito a contattare la struttura
Stato terminale	<code>409</code>

Alla cancellazione lo slot torna **immediatamente** prenotabile, e il token viene speso: un secondo tentativo con lo stesso link risponde `400`.

Da dove arriva il token. È il `MANAGE` emesso alla conferma (endpoint 4) o alla creazione di una prenotazione che nasce già confermata (endpoint 12). Arriva all'ospite nel link "*Gestisci o annulla la prenotazione*" dell'email di riepilogo.

⚠ Fino allo Step F questo endpoint era irraggiungibile. Cercava un token `CANCEL` o `MANAGE`, ma nessun percorso di codice ne emetteva uno: la rotta era scritta, testata a livello di servizio e documentata, e nessun client poteva procurarsi la credenziale richiesta. Il difetto è sopravvissuto perché i test del servizio costruivano il token a mano — cosa che un ospite non può fare. Vedi §8bis.11.3.

Email: `booking_cancelled`.

Test: `test_slot_liberato_dopo_la_cancellazione`,
`test_il_link_ricevuto_per_email_permette_davvero_di annullare`,
`test_il_link_di_gestione_si_spende_una_volta_sola`, `test_l annullamento_avvisa_l_ospite`.

6 • `POST /api/v1/bookings/lookup`

Consulta una prenotazione con codice ed email.

Accesso	Pubblico
Rate limit	10/ora per IP
Body	BookingLookupSchema
Risposta	BookingPublicSchema

Codici: 200 · 404 nessuna corrispondenza · 429

Logica. Richiede codice e email: il solo codice non basta, per evitare che tentativi a forza bruta espongano dati di altri ospiti. Il confronto sull'email è insensibile alle maiuscole; il codice viene normalizzato (spazi rimossi, maiuscolo).

Il 404 ha un messaggio **generico e identico** in ogni caso di fallimento: non deve lasciar capire se il codice esista e sia soltanto l'email a non corrispondere.

Test: test_lookup_con_dati_errati_non_rivela_nulla , test_lookup_con_codice_ed_email_corretti .

7 · GET /api/v1/bookings/me

Le prenotazioni dell'utente autenticato.

Accesso	Autenticato (cookie access_token)
Query	limit (1–100, default 20) · offset (≥ 0, default 0)
Risposta	List[BookingPublicSchema]

Codici: 200 · 401 sessione assente o scaduta

Ordinate per data di arrivo decrescente.

Test: test_le_mie_prenotazioni_richiedono_autenticazione .

8 · POST /api/v1/bookings/me

Crea una prenotazione come utente autenticato.

Accesso	Autenticato
Body	UserBookingCreateSchema
Risposta	BookingCreatedSchema — 201

Codici: 201 · 401 · 404 · 409 · 422

Logica. Identica alla creazione pubblica, con tre differenze: nessun captcha e nessun limite per email (la sessione autenticata è già una barriera contro l'automazione), e l'anagrafica **copiata dal profilo** invece che richiesta.

Lo snapshot anagrafico resta comunque congelato: una modifica successiva all'account non altera la prenotazione.

Test: test_creazione_utente_richiede_autenticazione ,
test_prenotazione_utente_autenticato_viene_persistita .

Quest'ultimo rilegge la prenotazione in una **richiesta successiva**, non solo nel corpo della risposta: è l'unico modo per accorgersi di un mancato commit. Vedi §11.2.

9 • `POST /api/v1/bookings/me/{code}/cancel`

Cancella una propria prenotazione.

Accesso	Autenticato
Path	<code>code</code> — codice prenotazione, es. <code>BB-2026-A7K3QX</code>
Body	<code>OwnBookingCancelSchema</code> — solo <code>reason</code> opzionale, nessun token
Risposta	<code>BookingPublicSchema</code>

Codici: `200` · `401` · `404` inesistente o di un altro utente · `409` non cancellabile

Si identifica col codice, non con l' `id` . `BookingPublicSchema` non espone l'identificativo interno, quindi un client non avrebbe modo di procurarselo: il codice è l'unico riferimento pubblico di una prenotazione, ed è quello che l'utente legge nella conferma.

Perché `404` e non `403` su una prenotazione altrui: distinguere i due casi confermerebbe l'esistenza del codice e consentirebbe di sondare le prenotazioni di altri utenti.

Test: `test_utente_cancella_la_propria_prenotazione` ,
`test_non_si_cancella_la_prenotazione_di_un_altro` .

4bis. Endpoint amministrativi

Tutte le rotte sotto `/api/v1/admin/bookings` richiedono un utente con `role = admin` . Senza cookie di sessione rispondono `401` , con un utente `role = user` rispondono `403` .

10 • `GET /api/v1/admin/bookings/`

Elenco filtrato e paginato.

Accesso	Amministrativo
Query	<code>status</code> (ripetibile) · <code>date_from</code> · <code>date_to</code> · <code>email</code> · <code>code</code> · <code>room_id</code> · <code>page</code> (default 1) · <code>page_size</code> (default 20, max 100)
Risposta	<code>PaginatedBookingsSchema</code>

Codici: `200` · `401` · `403` · `422` intervallo di date incoerente o `page_size` oltre il limite

Logica. Ordinate per data di arrivo decrescente, poi per creazione. `date_from` seleziona i soggiorni che **terminano dopo** quella data e `date_to` quelli che **iniziano prima**: insieme individuano i soggiorni che si sovrappongono all'intervallo, non solo quelli interamente contenuti.

Il filtro `status` si ripete per selezionarne più di uno: `?status=CONFIRMED&status=CHECKED_IN` .

Le righe camera sono caricate con `selectinload` : una query aggiuntiva per l'intera pagina, non una per prenotazione.

Test: `TestRead::test_elenco_paginato` , `test_filtro_per_stato` , `test_filtro_per_email` , `test_intervallo_di_date_invertito_rifiutato` .

```
curl -i -b cookies.txt "http://localhost:8000/api/v1/admin/bookings/?status=CONFIRMED&page=1&page_size=10"
```

11 • `GET /api/v1/admin/bookings/{booking_id}`

Dettaglio completo con timeline degli stati.

Accesso	Amministrativo
Risposta	<code>BookingSchema</code>

Codici: `200` · `401` · `403` · `404`

Logica. Vista completa: audit, canale di origine, note interne, riferimenti di pagamento e `status_history` con ogni transizione, attore e motivazione.

Il campo `version` restituito qui va rimandato nella `PATCH` . È quello che impedisce a due operatori di sovrascriversi a vicenda.

Test: `TestRead::test_dettaglio_con_storico` , `test_dettaglio_inesistente` .

12 • `POST /api/v1/admin/bookings/`

Creazione per conto di terzi.

Accesso	Amministrativo
Body	<code>AdminBookingCreateSchema</code>
Risposta	<code>AdminBookingCreatedSchema</code> — <code>201</code>

Codici: `201` · `401` · `403` · `404` utente o camera inesistenti · `409` slot occupato · `422` validazione

Logica. L'intestatario è un utente registrato (`user_id`) oppure un profilo inserito a mano (`guest`), mai entrambi né nessuno dei due.

Con `skip_email_confirmation` attivo — il default — la prenotazione nasce già `CONFIRMED` , senza token e senza blocco temporaneo: è il caso della prenotazione telefonica, dove l'identità è già stata verificata parlando con l'ospite. Disattivandolo si ottiene il flusso normale con conferma via email.

Non serve un preventivo firmato: l'admin è un attore fidato e il prezzo è calcolato dal server. Il percorso di creazione è però **lo stesso del canale pubblico** — lock, liberazione hold scaduti, verifica, exclusion constraint — quindi anche l'admin non può creare overbooking.

Sono consentite **date nel passato**, per registrare a posteriori un walk-in o correggere un errore.

⚠ **Frontend:** quando la data di arrivo è precedente a oggi, mostrare un dialog di conferma esplicito prima di inviare. Il backend lo consente di proposito, quindi l'unica difesa contro il refuso di digitazione è quell'avviso.

Email: `booking_confirmed` con il link di gestione quando nasce già confermata, `booking_pending` quando `skip_email_confirmation` è disattivato. Una prenotazione presa al telefono resta una prenotazione di cui l'ospite deve avere traccia scritta.

Test: l'intera classe `TestCreate` (7 test).

13 · `PATCH /api/v1/admin/bookings/{booking_id}`

Modifica di date, camere, ospiti, anagrafica e note.

Accesso	Amministrativo
Body	<code>AdminBookingUpdateSchema</code>
Risposta	<code>BookingSchema</code>

Codici: `200` · `401` · `403` · `404` · `409` version obsoleto, slot occupato, stato terminale · `422` validazione

Logica. Copre il caso più frequente del banco: l'ospite telefona per spostare il soggiorno o cambiare camera. Senza questa operazione l'unica via sarebbe cancellare e rifare, perdendo codice, storico e anagrafica.

1. verifica `version` → `409` se un altro operatore ha già salvato;
2. rifiuta se lo stato è terminale (`CANCELLED` , `EXPIRED` , `COMPLETED` , `NO_SHOW`);
3. se cambiano date o camere: lock, liberazione hold scaduti, verifica disponibilità **escludendo la prenotazione stessa**, ricalcolo del prezzo, ricostruzione delle righe camera;
4. aggiorna ospiti, anagrafica e note;
5. ricalcola il termine di cancellazione se le date sono cambiate;
6. registra la modifica nello storico con la motivazione.

L'esclusione al punto 3 non è un dettaglio: senza, una prenotazione che si allunga di un giorno colliderebbe con le proprie righe e si rifiuterebbe da sola.

L'opzione di pagamento non è modificabile qui: cambiarla altererebbe prezzo, politica di cancellazione e stato dell'incasso insieme. Se serve, si annulla e si ricrea.

Nessuna email. A differenza dell'annullamento, la modifica non avvisa l'ospite. Non è una dimenticanza: un messaggio di modifica deve dire cosa è cambiato rispetto a prima e se l'ospite debba fare qualcosa, e richiede il confronto fra stato precedente e successivo — un intervento a sé. Fino ad allora la comunicazione resta a chi opera al banco, che comunque sta già parlando con l'ospite.

Test: l'intera classe `TestUpdate` (8 test), fra cui `test_version_obsoleto_rifiutato` e `test_allungamento_di_un_giorno_non_collide_con_se_stessa` .

14 · `POST /api/v1/admin/bookings/{booking_id}/status`

Transizione di stato.

Accesso	Amministrativo
Body	<code>BookingStatusUpdateSchema</code>
Risposta	<code>BookingSchema</code>

Codici: 200 · 401 · 403 · 404 · 409 transizione non ammessa o incoerente con le date · 422 motivazione mancante sull'annullamento

Logica. Oltre alle transizioni della macchina a stati, valgono tre controlli temporali che intercettano i refusi più comuni del back-office:

Transizione	Vincolo
→ CHECKED_IN	non prima della data di arrivo
→ NO_SHOW	non prima che l'ospite fosse atteso
→ COMPLETED	non prima della data di partenza

L'annullamento libera **immediatamente** lo slot.

Email: solo l'annullamento genera un messaggio (`booking_cancelled`). Le altre transizioni riguardano il funzionamento interno della struttura — arrivo, partenza, mancata presentazione — e l'ospite le conosce già perché era presente. Un annullamento deciso al banco, invece, potrebbe non saperlo affatto.

Test: `TestOperations::test annullamento senza motivazione rifiutato` ,
`test_check_in_anticipato_rifiutato` , `test_transizione_illegale_rifiutata` ,
`test annullamento libera lo slot` .

15 · POST /api/v1/admin/bookings/{booking_id}/payment

Registrazione di un incasso manuale.

Accesso	Amministrativo
Body	<code>AdminPaymentRegistrationSchema</code>
Risposta	<code>BookingSchema</code>

Codici: 200 · 401 · 403 · 404 · 422

Logica. Per gli incassi in struttura: contanti, POS, bonifico. I pagamenti online passano dal webhook Stripe (Step G) e non vanno registrati da qui.

Test: `TestOperations::test_registrazione_incasso` .

16 · POST /api/v1/admin/bookings/{booking_id}/extend-hold

Proroga del blocco temporaneo.

Accesso	Amministrativo
Body	<code>BookingExtendHoldSchema</code> — <code>minutes</code> (1–120)
Risposta	<code>BookingSchema</code>

Codici: 200 · 401 · 403 · 404 · 409 prenotazione non in attesa, o camere nel frattempo vendute

Logica. Concede più tempo a un ospite che sta completando la prenotazione. Solo su prenotazioni in stato `PENDING_*` .

Se nel frattempo le camere sono state vendute a qualcun altro la proroga viene respinta: il blocco non si può riattivare su uno slot ormai occupato, e l'exclusion constraint lo impedisce.

Test: `TestOperations::test_proroga_hold_su_prenotazione_confermata_rifiutata` ,
`test_proroga_hold_su_prenotazione_in_attesa` .

17 • `POST /api/v1/admin/bookings/sweep-expired`

Esegue subito lo sweeper delle scadenze.

Accesso	Admin
Body	nessuno
Risposta	<code>SweepResultSchema</code>

Codici: `200` • `401` • `403`

Logica. Porta a `EXPIRED` le prenotazioni temporanee con blocco scaduto, disattiva le righe camera, invalida i token e scrive la traccia storica con attore `SYSTEM` . Le email partono **dopo** il commit, come nel giro automatico.

A cosa serve. A collaudare il meccanismo senza restare quindici minuti a guardare l'orologio, e come leva operativa quando lo sweeper in background è spento (`SWEeper_ENABLED=false`) perché lo si pilota da uno scheduler esterno.

Non è distruttivo. Libera slot che il sistema considera già liberi, e rieseguirlo non cambia nulla: la seconda passata non trova più prenotazioni in attesa scadute.

Test: `test_l_admin_puo_eseguire_lo_sweeper_a_mano` , `test_lo_sweeper_a_mano_e_idempotente` ,
`test_un_utente_semplice_non_puo_eseguire_lo_sweeper` ,
`test_senza_autenticazione_lo_sweeper_e_inaccessibile` .

```
curl -i -X POST http://localhost:8000/api/v1/admin/bookings/sweep-expired \  
-b cookies.txt
```

```
{"expired_count": 3, "notified_count": 2, "swept_at": "2026-09-20T14:31:07Z"}
```

5. Schemi

5.1 Richiesta

BOOKINGQUOTEREQUESTSCHEMA

Campo	Tipo	Obbl.	Vincoli
check_in	date	sì	≥ oggi, entro 365 giorni
check_out	date	sì	> check_in , da 1 a 30 notti
guest_count	int	sì	1–100
room_ids	UUID[]	sì	non vuota, senza duplicati, max 5
payment_option	enum	sì	PAY_NOW PAY_ON_ARRIVAL

GUESTDATASchema

Campo	Tipo	Obbl.	Vincoli
firstname	string	sì	2–50 caratteri, senza spazi esterni
lastname	string	sì	2–50 caratteri, senza spazi esterni
email	string	sì	email valida (RFC)
phone_number	string	sì	esattamente 10 cifre numeriche

GUESTBOOKINGCREATESchema

Campo	Tipo	Obbl.	Vincoli
quote_token	string	sì	20–4096 caratteri
guest	GuestDataSchema	sì	—
accept_terms	bool	sì	deve essere true
captcha_token	string	no	max 4096
website	string	no	honeypot: deve restare vuoto

USERBOOKINGCREATESchema

Campo	Tipo	Obbl.
quote_token	string	sì
accept_terms	bool	sì (deve essere true)

BOOKINGCONFIRMSchema · BOOKINGCANCELSCHEMA · OWNBOOKINGCANCELSCHEMA · BOOKINGLOOKUPSCHEMA

Schema	Campi
BookingConfirmSchema	token (string, 20–512, obbligatorio)
BookingCancelSchema	token (obbligatorio) · reason (max 500, opzionale)
OwnBookingCancelSchema	reason (max 500, opzionale) — nessun token: basta la sessione
BookingLookupSchema	code (3–20, normalizzato in maiuscolo) · email

SCHEMI AMMINISTRATIVI (ENDPOINT ALLO STEP E)

Schema	Campi principali
AdminBookingCreateSchema	date, guest_count, room_ids, payment_option, payment_method?, user_id XOR guest, skip_email_confirmation (default true), mark_as_paid (default false), admin_notes? — consente date nel passato
AdminBookingUpdateSchema	version (obbligatorio), date, guest_count, room_ids, guest?, admin_notes?, reason? — consente date nel passato; l'opzione di pagamento non è modificabile
BookingStatusUpdateSchema	new_status, reason? — obbligatoria se new_status = CANCELLED
AdminPaymentRegistrationSchema	payment_method, payment_status, amount?
BookingExtendHoldSchema	minutes (1–120)
BookingSearchFiltersSchema	status[]?, date_from?, date_to?, email?, code?, room_id?, page (default 1), page_size (default 20, max 100)

Nessuno schema di input contiene un campo prezzo. C'è un test che lo verifica.

5.2 Risposta

AVAILABILITYRESPONSESCHEMA

Campo	Tipo	Note
check_in / check_out	date	—
nights	int	notti di soggiorno
guest_count	int	—
rooms	AvailableRoomSchema[]	ordinate per numero camera
suggested_combinations	RoomCombinationSchema[]	max 10, minimali

AVAILABLEROOMSCHEMA

Campo	Tipo	Note
id · name · number · capacity	UUID, string, int, int	—
price_per_night	decimal string	prezzo a notte
nights · subtotal	int, decimal string	subtotale per l'intero soggiorno
services	RoomServiceSchema[]	{id, name}
fits_all_guests	bool	la camera da sola basta per gli ospiti

ROOMCOMBINATIONSCHEMA

room_ids (UUID[]) · rooms_count (int) · total_capacity (int) · total_price (decimal string) · wasted_capacity (int, posti letto eccedenti)

Porta solo gli identificativi: i dati delle camere sono già in rooms .

BOOKINGQUOTERESPONSESCHEMA

check_in · check_out · nights · guest_count · payment_option · lines[] · base_price · discount_amount · total_price · currency · quote_token · quote_expires_at

lines[] è PriceLineSchema : room_id , room_name , unit_price , nights , line_total .

BOOKINGPUBLICSCHEMA — VISTA PER L'OSPITE

Campo	Tipo	Note
code	string	codice leggibile, es. BB-2026-A7K3QX
status	enum	vedi §6
check_in · check_out · nights	date, date, int	—
guest_count	int	—
guest_firstname · guest_lastname · guest_email	string	snapshot anagrafico
rooms	BookingRoomItemSchema[]	—
base_price · discount_amount · total_price · currency	decimal string	—
payment_option · payment_status	enum	—
hold_expires_at	datetime null	scadenza del blocco temporaneo
cancellation_deadline	datetime null	termine di cancellazione gratuita
confirmed_at	datetime null	—

Non contiene `id` , `user_id` , `version` , `admin_notes` , campi di audit né riferimenti Stripe. C'è un test che lo verifica.

BOOKINGROOMITEMSCHEMA

`room_id` · `room_name` · `room_number` · `check_in` · `check_out` · `nights` · `unit_price` · `line_total`

`unit_price` è il prezzo **congelato** al momento della prenotazione: una modifica successiva al listino non lo altera.

BOOKINGCREATEDSCHEMA

Campo	Tipo	Note
<code>booking</code>	<code>BookingPublicSchema</code>	—
<code>confirmation_token</code>	string null	solo se <code>EMAIL_ENABLED=false</code> — vedi §3.6

BOOKINGSHEMA — VISTA AMMINISTRATIVA

Tutti i campi di `BookingPublicSchema` , più: `id` , `source_channel` , `user_id` , `guest_phone` , `payment_method` , `cancelled_at` , `cancellation_reason` , `admin_notes` , `created_at` , `updated_at` , `created_by` , `last_updated_by` , `version` , `status_history[]` .

`version` è il contatore dell'**optimistic locking**. Il frontend lo legge con la `GET` e lo rimanda invariato nella `PATCH` . Se nel frattempo un altro operatore ha salvato, la risposta è `409` e la sua modifica non viene sovrascritta. Volutamente assente da `BookingPublicSchema` : all'ospite non serve.

ADMINBOOKINGCREATEDSCHEMA

`booking` (`BookingSchema`) · `confirmation_token` (string | null).

Porta la vista completa, non quella pubblica: l'admin deve vedere audit, canale di origine e note interne. Il token è `null` quando `skip_email_confirmation` è attivo.

BOOKINGSTATUSHISTORYSCHEMA

`from_status?` · `to_status` · `actor_type` · `reason?` · `created_at` .

Lo storico registra **anche le modifiche che non cambiano stato**: una `PATCH` produce una riga con `from_status == to_status` e una descrizione di cosa è cambiato. "Chi ha spostato le date" è esattamente l'informazione che serve in caso di contestazione.

BOOKINGLISTITEMSCHEMA E PAGINATEDBOOKINGSSHEMA

Riga di elenco: `id` , `code` , `status` , `check_in` , `check_out` , `guest_lastname` , `guest_email` , `rooms_count` , `total_price` , `payment_status` . Contenitore: `items[]` , `total` , `page` , `page_size` , `pages` .

SWEEPRESULTSCHEMA

Campo	Tipo	Note
<code>expired_count</code>	int	Prenotazioni portate a <code>EXPIRED</code>
<code>notified_count</code>	int	Ospiti avvisati via email
<code>swept_at</code>	datetime	Istante di esecuzione, in UTC

`notified_count` è minore o uguale a `expired_count` : le scadenze più vecchie di `SWEEPER_NOTIFY_MAX_AGE_HOURS` vengono sistemate a database ma non notificate.

6. Enumerazioni

BookingStatus

Valore	Significato	Slot occupato
<code>PENDING_CONFIRMATION</code>	In attesa di conferma via email	sì, finché il blocco è valido
<code>PENDING_PAYMENT</code>	In attesa del pagamento online	sì, finché il blocco è valido
<code>CONFIRMED</code>	Confermata	sì
<code>CHECKED_IN</code>	Ospite arrivato	sì
<code>COMPLETED</code>	Soggiorno concluso	sì
<code>CANCELLED</code>	Annullata (<i>terminale</i>)	no
<code>EXPIRED</code>	Blocco scaduto senza conferma (<i>terminale</i>)	no
<code>NO_SHOW</code>	Ospite non presentato (<i>terminale</i>)	sì

Transizioni ammesse

Da	A
<code>PENDING_CONFIRMATION</code>	<code>CONFIRMED</code> · <code>EXPIRED</code> · <code>CANCELLED</code>
<code>PENDING_PAYMENT</code>	<code>CONFIRMED</code> · <code>EXPIRED</code> · <code>CANCELLED</code>
<code>CONFIRMED</code>	<code>CHECKED_IN</code> · <code>CANCELLED</code> · <code>NO_SHOW</code>
<code>CHECKED_IN</code>	<code>COMPLETED</code>
<code>COMPLETED</code> · <code>CANCELLED</code> · <code>EXPIRED</code> · <code>NO_SHOW</code>	(<i>terminali</i>)

Ogni transizione non prevista produce `409` . Ogni transizione eseguita lascia una riga in `booking_status_history` con attore e motivazione.

Altri enum

Enum	Valori
PaymentOption	PAY_NOW · PAY_ON_ARRIVAL
PaymentStatus	NOT_REQUIRED · PENDING · AUTHORIZED · PAID · FAILED · REFUNDED · PARTIALLY_REFUNDED
PaymentMethod	STRIPE_CARD · CASH_ON_SITE · POS_ON_SITE · BANK_TRANSFER
BookingChannel	PUBLIC_GUEST · PUBLIC_USER · ADMIN_BACKOFFICE
BookingTokenPurpose	CONFIRM_EMAIL · MANAGE · CANCEL
AuditActorType	GUEST · USER · ADMIN · SYSTEM
UserRole	user · admin

7. Copertura dei test

`pytest` dalla root del progetto. Il database di test si crea da solo alla prima esecuzione.

File	Test	Database	Cosa verifica
<code>test_booking_schema.py</code>	39	no	Vincoli dei DTO: date, capienza, XOR utente/ospite, honeypot, normalizzazione codice
<code>test_pricing_service.py</code>	17	no	Sconti, arrotondamento <code>ROUND_HALF_UP</code> , assenza di <code>float</code> , quote token, penali
<code>test_availability_combinations.py</code>	11	no	Minimalità, ordinamento, limiti, euristica su inventari ampi
<code>test_booking_service.py</code>	14	sì	Concorrenza (eseguita 10 volte) , ciclo di vita, transizioni, hold scaduto, back-to-back
<code>test_booking_api.py</code>	26	sì	Flusso end-to-end, rate limit, protezioni, autorizzazione, persistenza, email
<code>test_admin_booking_api.py</code>	32	sì	Autorizzazione, creazione on-behalf-of, modifica con optimistic locking, stato, incassi
<code>test_email_service.py</code>	17	no	Rendering dei template, escaping, link, mascheramento nei log, robustezza del canale
<code>test_booking_expiration.py</code>	15	sì	Transizione a <code>EXPIRED</code> , slot riprenotabile, idempotenza, soglia di notifica, endpoint admin
Totale eseguito	~172		il test di concorrenza è parametrizzato su 10 iterazioni

Il test che conta più di tutti

`test_due_prenotazioni_concorrenti_una_sola_vince` lancia due creazioni identiche con `asyncio.gather` su sessioni distinte e pretende **esattamente un successo e un `RoomNotAvailable`**. È ripetuto 10 volte, perché una race condition che si manifesta una volta su dieci resta una race condition.

L'anti-overbooking non è garantito dal codice applicativo ma da un **exclusion constraint di PostgreSQL**: nessuna sequenza di operazioni concorrenti può produrre una doppia vendita.

Il test che ha trovato un bug scrivendosi

`test_il_link_ricevuto_per_email_permette_davvero_di annullare` estrae il token dal corpo dell'email e lo usa su `POST /cancel`, esattamente come farebbe l'ospite. È l'unico test che percorre l'intera catena *emissione* → *email* → *endpoint*, ed è quello che ha reso visibile un endpoint che nessun client poteva raggiungere (§8bis.11.3).

La lezione è generale: un test che chiama il servizio direttamente dimostra che il servizio parla con sé stesso. Solo un test che si mette nei panni del chiamante dimostra che il chiamante può arrivarci.

8. Checklist di verifica manuale

Da eseguire su Swagger (`/docs`) a sviluppo concluso.

Prerequisiti

- [] Almeno due camere abilitate in `rooms`
- [] Un utente con `role = admin`
- [] `EMAIL_ENABLED=false` (per ricevere il token in risposta)

Flusso ospite — percorso felice

- [] `GET /availability` restituisce le camere libere con `fits_all_guests` corretto
- [] Le combinazioni proposte coprono gli ospiti e non contengono camere superflue
- [] `POST /quote` restituisce un totale coerente e un `quote_token`
- [] `POST /` crea la prenotazione in `PENDING_CONFIRMATION` e restituisce il token
- [] `POST /confirm` porta a `CONFIRMED`, azzera `hold_expires_at`, valorizza `cancellation_deadline`
- [] `POST /lookup` con codice ed email restituisce la prenotazione

Flusso ospite — casi limite

- [] Ripetere `POST /` sullo stesso slot → `409`
- [] Confermare due volte → `409` "già stata confermata"
- [] `POST /` con `website` valorizzato → `422`
- [] `POST /` con `accept_terms: false` → `422`
- [] `POST /quote` con `check_out` precedente a `check_in` → `422`
- [] `POST /quote` con più ospiti della capienza → `422`
- [] Ventun richieste a `/availability` in un minuto → `429` con `Retry-After`
- [] `POST /lookup` con email errata → `404` con messaggio generico

Autorizzazione

- [] `GET /me` senza cookie → `401`
- [] `GET /me` con cookie valido → le proprie prenotazioni
- [] `POST /me` come utente autenticato → la prenotazione compare in `GET /me`

- [] Cancellare la propria prenotazione col codice → `CANCELLED`
- [] Cancellare la prenotazione di un altro utente → `404`

Area amministrativa

- [] `GET /admin/bookings/` senza cookie → `401` ; con utente normale → `403`
- [] Creazione per conto di terzi con ospite manuale → nasce `CONFIRMED` , nessun token
- [] Creazione con `user_id` → anagrafica presa dal profilo
- [] Creazione con `user_id` e `guest` insieme → `422`
- [] `PATCH` che sposta le date → prezzo ricalcolato, righe camera riallineate
- [] `PATCH` con `version` obsoleto → `409` "modificata da un altro operatore"
- [] `PATCH` verso uno slot occupato → `409`
- [] `PATCH` su prenotazione annullata → `409`
- [] Check-in registrato prima della data di arrivo → `409`
- [] Annullamento senza motivazione → `422`
- [] Annullamento → lo slot torna immediatamente prenotabile
- [] Registrazione incasso in contanti → `payment_status: PAID`
- [] Proroga hold su prenotazione confermata → `409`

Integrità dei dati

- [] Modificare il prezzo di una camera dopo una prenotazione → il totale storico resta invariato
- [] Cancellare una camera con prenotazioni → `409 EntityInUse`
- [] Cancellare un utente → le sue prenotazioni sopravvivono con `user_id` a `null`

Email

- [] `POST /` → nei log compare l'invio di `booking_pending` con il link di conferma
- [] Seguire il link e confermare → arriva `booking_confirmed` con il **link di gestione**
- [] Usare quel link su `POST /cancel` → la prenotazione passa a `CANCELLED`
- [] Riusare lo stesso link → `400` : il token si spende una volta sola
- [] L'annullamento genera `booking_cancelled`
- [] Con `EMAIL_ENABLED=true` , `confirmation_token` è `null` nella risposta
- [] All'avvio di uvicorn compare `Sweeper avviato: intervallo ... secondi` — se manca, i log applicativi non hanno una destinazione
- [] Prenotare con un nome contenente `<b>test</b>` → nell'HTML compare escapato, non interpretato
- [] Spegnerne il server SMTP e prenotare → la prenotazione nasce comunque (`201`)

Sweeper delle scadenze

- [] Attesa oltre 15 minuti senza conferma → stato `EXPIRED` e slot riprenotabile
- [] `POST /admin/bookings/sweep-expired` → `expired_count` coerente
- [] Rieseguirlo subito dopo → `expired_count: 0`
- [] Dopo lo sweep, riprenotare le stesse date sulla stessa camera → riesce
- [] Il link di conferma di una prenotazione scaduta non funziona più
- [] Nel dettaglio admin, l'ultima riga di storico ha attore `SYSTEM`

- [] Con `SWEeper_ENABLED=false` nulla scade da solo, ma l'endpoint 17 funziona

8bis. Tre trappole da conoscere

Emerse durante lo sviluppo, e rilevanti per chi lavora su questo codice.

11.1 Un errore di validazione può essere colpa del server

L'handler globale è registrato sulla `ValidationError` di Pydantic, che viene sollevata sia quando il **client** manda dati sbagliati, sia quando è il **server** a costruire male un modello di risposta. Entrambi i casi producono oggi un `422` con un messaggio del tipo *"Il campo X è obbligatorio"*.

Se ricevi un `422` che nomina un campo che non hai inviato — e che non appartiene alla richiesta — non stai sbagliando tu: è un bug del backend nella costruzione della risposta. *(Correzione in coda al debito tecnico: gli errori di costruzione lato server dovranno diventare `500`.)*

11.2 Un `201` non dimostra che il dato sia stato salvato

SQLAlchemy 2.0 apre una transazione alla **prima query, anche di sola lettura**. La dipendenza di autenticazione legge la tabella utenti prima di entrare nell'endpoint, quindi su ogni rotta protetta la sessione risulta già "in transazione".

Il Service concludeva perciò che il commit spettasse a qualcun altro, e la scrittura non veniva mai persistita: risposta `201` con il corpo corretto, database vuoto. Corretto allo Step E chiudendo la transazione esplicitamente.

Conseguenza per i test: verificare una scrittura rileggendo il corpo della risposta non prova nulla. Serve una **richiesta successiva**, che usa una sessione diversa.

11.3 Un endpoint può essere corretto e comunque irraggiungibile

Due casi nello stesso modulo, trovati a distanza di un giorno.

`POST /me/{booking_id}/cancel` chiedeva l' `id` interno, che `BookingPublicSchema` non espone per scelta: nessuna risposta lo restituiva all'ospite, quindi nessun client poteva costruire quell'URL. Corretto allo Step E in `/me/{code}/cancel`.

`POST /cancel` cercava un token `CANCEL` o `MANAGE`, e **nessun percorso di codice ne emetteva uno**. La rotta era scritta, coperta da test di servizio e documentata; semplicemente, la credenziale che pretendeva non esisteva. Corretto allo Step F emettendo il token `MANAGE` alla conferma.

Entrambi sono sopravvissuti ai test perché quei test chiamavano il servizio direttamente, costruendosi in casa ciò che un client non può costruire. E in entrambi i casi non è stato un test a trovarli: è stato il **doverli scrivere**, cioè il primo momento in cui qualcuno si è messo nei panni di chi chiama.

Due controlli che vale la pena fare quando si aggiunge una rotta pubblica:

1. ogni parametro di percorso compare in almeno uno schema di risposta pubblico;
 2. ogni credenziale richiesta viene emessa da qualche parte, e quel punto è raggiungibile dall'utente che dovrà spenderla.
-

9. Changelog

Data	Step	Modifiche
20/09/2026	F	<code>configure_logging()</code> : i log applicativi avevano un logger ma nessun handler, quindi venivano scartati in silenzio · servizio email con 4 template HTML+testo · sweeper delle scadenze in <code>lifespan</code> · <code>POST /admin/bookings/sweep-expired</code> · token <code>MANAGE</code> finalmente emesso: <code>POST /cancel</code> era irraggiungibile · invio post-commit con <code>BackgroundTasks</code> · <code>FOR UPDATE SKIP LOCKED</code> sullo sweeper · nessuna email su <code>PENDING_PAYMENT</code>
20/09/2026	E	7 endpoint amministrativi · <code>POST /me/{code}/cancel</code> ora usa il codice invece dell' <code>id</code> interno (era inutilizzabile dal client) · modifica di date e camere con ricalcolo prezzo · optimistic locking esposto al client (<code>version</code>) · <code>ConcurrentModification</code> · storico esteso alle modifiche non di stato
20/09/2026	D	9 endpoint pubblici e utente · rate limiting · captcha Turnstile · honeypot · <code>Retry-After</code> · <code>BookingCreatedSchema</code> · <code>trusted_proxy_count</code>
20/09/2026	C	Preventivo firmato · token monouso · motore di prenotazione con locking a 3 livelli · disponibilità con combinazioni
19/09/2026	B	21 DTO · 3 repository · <code>AppException</code> con handler unico · correzione 500 → 422 sulle validazioni · <code>EntityInUse</code>
19/09/2026	A	Modelli e macchina a stati · exclusion constraint anti-overbooking · foreign key composita · guest booking

10. Da completare

Step	Contenuto	Impatto su questo documento
G	Pagamenti Stripe	Endpoint webhook e creazione Payment Intent; codici <code>402</code> ; l'email di conferma per le prenotazioni <code>PAY_NOW</code> partirà da lì
—	Email di modifica	Un messaggio che dica cosa è cambiato quando l'admin sposta date o camere (endpoint 13)
—	Outbox pattern	Oggi un'email persa fra commit e invio non lascia traccia. Con un outbox l'invio diventa ritentabile

Documento generato automaticamente dal sorgente `docs/api/booking_api.md` . Per segnalare una discrepanza fra documento e codice, fa fede il codice.